代码块

```cpp
#include <iostream>

using namespace std;

typedef long long LL;

// 加法比乘法快，而且防溢出
LL qmul(LL a, LL b, LL p)
{
    LL sum = 0;
    while(b) // 枚举 b 的二进制位
    {
        if(b & 1) sum = (sum + a) % p;
        a = (a + a) % p; // 计算下一个权值
        b >>= 1;
    }
    return sum % p;
}

int main()
{
    LL a, b, p;
    cin >> a >> b >> p;
    cout << qmul(a % p, b, p) << endl;

    return 0;
}
```

10. 离散化

【算法原理】

当题目中数据范围很大，但是数据的总量不是很大，并且我们需要用数据的值来映射数组的下标时，我们就可以用离散化的思想先预处理一下所有的数据，使的每一个数据都映射成一个范围较小的值。

比如：$[9, 99, 9999, 999999]$ 离散之后就变成 $[1, 2, 3, 4]$。

【离散化模板一】

代码块

```cpp
// 离散化方式一：排序 + 去重 + 二分查找离散化后的值
#include <iostream>
```

```cpp
#include <algorithm>

using namespace std;

const int N = 1e5 + 10;

int n;
int a[N]; // 原始数据

int pos; // 记录离散化数组中元素的个数
int disc[N]; // 离散化需要的数组

// 二分查找离散化之后的值，其实就是排序之后的下标
int find(int x)
{
    int l = 1, r = pos; // 注意查找的区间
    while(l < r)
    {
        int mid = (l + r) >> 1;
        if(disc[mid] >= x) r = mid;
        else l = mid + 1;
    }
    return l;
}

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++)
    {
        int x; cin >> x;
        a[i] = x;
        disc[++pos] = x; // 数据放进离散化数组中
    }

    // 离散化：排序 + 去重
    sort(disc + 1, disc + 1 + pos);
    pos = unique(disc + 1, disc + 1 + pos) - (disc + 1);

    // 找到离散化之后的值
    for(int i = 1; i <= n; i++)
    {
        int x = a[i];
        cout << x << "离散化之后是：  " << find(x) << endl; // 二分查找离散化之后的值
    }

    return 0;
```

```
}
```

【离散化模版二】

代码块

```
// 离散化方式二：排序 + STL
// 本质是和方式一一样的，只不过借助了 STL，去重以及查找更方便
#include <iostream>
#include <unordered_map>
#include <algorithm>

using namespace std;

const int N = 1e5 + 10;

int n;
int a[N]; // 原始数据

int tmp[N]; // 用来排序的数组
int cnt;
unordered_map<int, int> id; // 记录离散化之后的值

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++)
    {
        int x; cin >> x;
        a[i] = x;
        tmp[i] = x; // 数据放进离散化数组中
    }

    // 离散化：排序 + 放进哈希表中
    sort(tmp+ 1, tmp + 1 + n);
    for(int i = 1; i <= n; i++)
    {
        if(id.count(tmp[i])) continue; // 如果已经存过这个数，不做处理
        cnt++; // 这个数映射之后的值
        id[tmp[i]] = cnt; // 放进哈希表中
    }

    // 找到离散化之后的值
    for(int i = 1; i <= n; i++)
    {
```

```
40          int x = a[i];
41          cout << x << "离散化之后是:  " << id[a[i]] << endl; // 二分查找离散化之后的
    值
42      }
43
44      return 0;
45  }
```

【注意事项】

1. 离散化是一种「思想」，模版其实不用背，根据算法思想就可以实现。而且实现离散化的方式也可以在上述模板的基础上「修改」，千万「不要生搬硬套」（大家也会看到有些题解里面是借助「结构体」离散化的，但是核心的思想都是不变的）；
2. 前期学习离散化的时候可能会被「绕」进去，会把「离散前」和「离散后」的值搞混，分不清楚是用离散前的值还是离散后的值。觉得迷惑是「很正常」，一定要根据最基础的模板画图分析整个流程，搞清楚每一个变量的作用，以及达到的目的。离散化「使用的多」了，慢慢的就不会迷了。

10.1 火烧赤壁

> **题目来源：** 洛谷
>
> **题目链接：** P1496 火烧赤壁
>
> **难度系数：** ★★

【题目描述】

> 曹操平定北方以后，公元 208 年，率领大军南下，进攻刘表。他的人马还没有到荆州，刘表已经病死。他的儿子刘琮听到曹军声势浩大，吓破了胆，先派人求降了。孙权任命周瑜为都督，拨给他三万水军，叫他同刘备协力抵抗曹操。
>
> 隆冬的十一月，天气突然回暖，刮起了东南风。没想到东吴船队离开北岸大约二里距离，前面十条大船突然同时起火。火借风势，风助火威。十条火船，好比十条火龙一样，闯进曹军水寨。那里的船舰，都挤在一起，又躲不开，很快地都烧起来。一眨眼工夫，已经烧成一片火海。
>
> 曹操气急败坏的把你找来，要你钻入火海把连环线上着火的船只的长度统计出来！
>
> 给定每个起火部分的起点和终点，请你求出燃烧位置的长度之和。

【输入描述】

> 第一行一个整数，表示起火的信息条数 n 。
> 接下来 n 行，每行两个整数 a,b ，表示一个着火位置的起点和终点（注意：左闭右开）。

> 对于全部的测试点，保证 $1 \leq n \leq 2 \times 10^4$，$-2^{31} \leq a < b < 2^{31}$ ，且答案小于 2^{31} 。

【输出描述】

> 输出一行一个整数表示答案

【示例一】

> 输入：
>
> 3
>
> -1 1
>
> 5 11
>
> 2 9
>
> 输出：
>
> 11

【解法】

抛开数据范围不看，这就是一道「差分」题目：

- 给定一个区间，我们可以全部执行 $+1$ 操作；
- 最后看看整个数组中，大于 0 的位置有多少个。

因此可以创建一个原数组的「差分」数组，然后执行完「区间修改」操作之后，还原原数组，「统计大于 0 」的区间长度。

但是，这道题的「数据范围」不允许我们直接差分，因为「开不了那么大」的数组；即使能开那么大的数组，「时间」也不够用。

我们发现，区间的范围虽然很大，区间的「个数」却只有 2×10^4 级别。此时我们就可以：

1. 先将所有的「区间信息」离散化；
2. 然后在「离散化的基础」上，处理所有的「区间修改」操作；
3. 处理完之后找出「原始数组对应的区间端点」，计算相应的「长度」。

【参考代码】

代码块

```
#include <iostream>
#include <algorithm>

using namespace std;

```

```cpp
const int N = 2e4 + 10;

int n;
int l[N], r[N];

int m; // 离散数组的大小
int disc[N * 2]; // 离散之后的数组
int f[N * 2]; // 差分数组

int find(int x)
{
    int l = 1, r = m;
    while(l < r)
    {
        int mid = (l + r + 1) >> 1;
        if(disc[mid] <= x) l = mid;
        else r = mid - 1;
    }
    return l;
}

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++)
    {
        int x, y; cin >> x >> y;
        l[i] = x, r[i] = y;
        disc[++m] = l[i], disc[++m] = r[i];
    }

    // 离散化处理
    sort(disc + 1, disc + 1 + m);
    m = unique(disc + 1, disc + 1 + m) - disc - 1;

    // 处理区间修改
    for(int i = 1; i <= n; i++)
    {
        int x = find(l[i]), y = find(r[i]);
        f[x] += 1, f[y] -= 1;
    }

    // 还原数组
    for(int i = 1; i <= m; i++) f[i] += f[i - 1];

    // 找出每一段大于0的区间，统计长度
    int sum = 0;
```

```
    for(int i = 1; i <= m; i++)
    {
        if(f[i] <= 0) continue;
        int j = i;
        while(j <= m && f[j] > 0) j++;
        // 累加长度的时候记得使用离散化之前的值
        sum += disc[j] - disc[i];
        i = j;
    }
    cout << sum << endl;

    return 0;
}
```

10.2 贴海报

题目来源： 洛谷

题目链接： P3740 [HAOI2014] 贴海报

难度系数： ★★

【题目描述】

Bytetown 城市要进行市长竞选，所有的选民可以畅所欲言地对竞选市长的候选人发表言论。为了统一管理，城市委员会为选民准备了一个张贴海报的 electoral 墙。

张贴规则如下：

1. electoral 墙是一个长度为 N 个单位的长方形，每个单位记为一个格子；
2. 所有张贴的海报的高度必须与 electoral 墙的高度一致的；
3. 每张海报以 `A B` 表示，即从第 A 个格子到第 B 个格子张贴海报；
4. 后贴的海报可以覆盖前面已贴的海报或部分海报。

现在请你判断，张贴完所有海报后，在 electoral 墙上还可以看见多少张海报。

【输入描述】

第一行，两个正整数 N, M ，分别表示 electoral 墙的长度和海报个数。

接下来 M 行，每行两个正整数 A_i, B_i ，表示每张海报张贴的位置。

数据范围：$10 \leq N \leq 10000000, 1 \leq M \leq 1000, 1 \leq A_i, B_i \leq 10000000$

【输出描述】

输出贴完所有海报后，在 electoral 墙上还可以看见的海报数。

【示例一】

输入：

100 5

1 4

2 6

8 10

3 4

7 10

输出：

4

【解法】

根据题意「模拟」即可。

由于「区间的长度」很大，暴力模拟的时候会超时。但是我们发现，虽然区间长度很大，但是「区间的个数」是很少的，所以我们可以「离散化」处理一下区间的端点值，然后在「离散化的基础上」模拟覆盖情况。

注意注意注意，重要的注意说三遍：

离散化在离散「区间问题」的时候一定要小心！因为我们离散化操作会把区间缩短，从而导致丢失一些点。在涉及「区间覆盖」问题上，离散化会导致「结果出错」。

比如我们这道题，如果有三个区间分别为：$[2,5],[2,3],[5,6]$，离散化之后为：$[1,3],[1,2],[3,4]$，区间覆盖如图所示：



为了避免出现上述情况，我们可以在离散化的区间 $[x,y]$ 时，不仅考虑 x,y 这两个值，也把「$x+1,y+1$」也考虑进去。此时「单个区间内部」就出现空隙，「区间与区间之间」也会出现空隙。就可以避免上述情况出现。

可见，离散化之后可能会导致结果错误，使用的时候还是需要「谨慎」一点。

【参考代码】

代码块

```
#include <iostream>
#include <algorithm>

using namespace std;

const int N = 1010;

int n, m;
int a[N], b[N];

int pos;
int disc[N * 4]; // 因为有两套位置

int w[N * 4];
bool mp[N];

// 找到 x 映射之后的数，也就是 x 的下标
int find(int x)
{
    int l = 1, r = pos;
    while(l < r)
    {
        int mid = (l + r) >> 1;
        if(disc[mid] >= x) r = mid;
        else l = mid + 1;
    }
    return l;
}

int main()
{
    cin >> n >> m;
    for(int i = 1; i <= m; i++)
    {
        int x, y; cin >> x >> y;
```

```
        a[i] = x, b[i] = y;
        // 离散化之后有可能导致区间缩小，多加一个位置
        disc[++pos] = x, disc[++pos] = x + 1;
        disc[++pos] = y, disc[++pos] = y + 1;
    }

    // 离散化
    sort(disc + 1, disc + 1 + pos);
    pos = unique(disc + 1, disc + 1 + pos) - (disc + 1);

    // 用离散化之后的值覆盖区间
    for(int i = 1; i <= m; i++)
    {
        int x = find(a[i]), y = find(b[i]);
        for(int j = x; j <= y; j++) w[j] = i;
    }

    // 统计整个数组中，一共有多少个不同的数
    int cnt = 0;
    for(int i = 1; i <= pos; i++)
    {
        int x = w[i];
        if(!x) continue; // 不要统计 0
        if(mp[x]) continue;
        cnt++;
        mp[x] = true;
    }

    cout << cnt << endl;

    return 0;
}
```

11. 递归初阶

递归：函数自己调用自己。

在本小结里，我们会通过一些简单题目了解递归的本质。先带着大家消除对递归的恐惧，为我们后续学习更高级的算法做铺垫。

11.1 汉诺塔

> **题目来源：** 信息学奥赛一本通
>
> **题目链接：** 汉诺塔问题